DRAFT SPECIFICATION — FOR REVIEW

ArkFloat: Liquidity Protocol for Ark Service Providers

Float capital. Float the Ark.

Version: 0.3 (Draft)

Date: March 2026

Status: Specification for review

Published by: thebitcoinbreakdown.com

Changelog — v0.3: revised LP security framing, corrected protocol change scope, expanded exit risk section, removed unverified design directions, updated open questions based on source code review.

ABSTRACT

ArkFloat is a protocol proposal for a BTC-native lending system that funds Ark rounds with external capital. Ark Service Providers (ASPs) face a structural capital constraint: every round they run locks BTC into a covenant tree for 7–30 days, and new rounds require fresh capital while prior trees have not yet expired. ArkFloat addresses this by enabling external liquidity providers (LPs) to contribute BTC directly into Ark rounds. LP repayment is secured by the round's own forfeit mechanism — the same pre-signed Bitcoin transactions that protect ASP capital — routed to the LP's address instead. No custodian, no oracle, no new trust assumptions. LP recovery does depend on user refresh behavior rather than ASP solvency; this risk is detailed in Section 6.

CONTENTS

1. Introduction
2. The Problem
3. The Cause
4. The Solution
5. Impact
6. Viability

7. Risks and Failure Modes
8. Open Questions
9. References

---

1. Introduction

Ark is a Bitcoin Layer 2 protocol where a central server (the ASP) locks BTC into shared UTXO trees on behalf of users. Users hold virtual UTXOs (VTXOs) — off-chain claims within those trees — and transact by participating in rounds or sending out-of-round (OOR) payments. VTXOs expire after a configurable period, at which point the ASP recovers the locked BTC.

Ark needs capital to grow. Every round an ASP runs locks BTC into a covenant tree for 7–30 days. The next round needs fresh BTC while the last round's capital is still frozen. Run enough rounds and you outgrow your own balance sheet.

Today, that ceiling is the binding constraint on how many ASPs can operate and at what scale. A well-run ASP with 300 BTC hits the wall at 230 BTC/week throughput (7-day expiry) or 55 BTC/week (30-day expiry). To go further, they either shorten expiry (worse user experience), stop growing, or find outside capital.

ArkFloat is a proposal for an Ark-native lending system that funds rounds with external capital. External liquidity providers contribute BTC directly into Ark rounds. Their repayment is secured by the round's own forfeit mechanism — the same pre-signed Bitcoin transactions that already protect the ASP's capital — routed to the LP's address instead. No custodian. No oracle. No added counterparty risk. ArkFloat is a lending protocol between Ark operators and external capital providers, built directly into Ark's round construction.

For LPs, this creates something that barely exists in Bitcoin: a non-custodial, fixed-term yield product. The LP's BTC goes into an Ark round, earns 1–3% annualized, and is returned at expiry whether the ASP is operational or not. The security comes from Bitcoin script, not from trusting the ASP to pay up.

For the Ark ecosystem, it removes the "only Tether can afford to run an ASP" problem. Smaller operators can borrow the capital gap, grow with demand, and compete on service quality rather than balance sheet size.

Ark exists to bring Bitcoin to people who don't have a UTXO — the unbanked, the underserved, anyone priced out of on-chain fees. But the Ark is fully reserved by design: every satoshi an ASP serves must be

locked on-chain first. That's what makes it trustless. It's also what limits how many people can board. People who already hold UTXOs can change that. By lending idle capital to ASPs, they float the operators who serve the people who have nothing yet. The capital circulates. The Ark grows. More people board before the tide rises.

The mechanism works by extending what Ark already does. It does not add a new trust layer — it redirects an existing one. The open design problem is LP exposure to unilateral exit risk: if users in a funded round exit on-chain rather than refreshing, those forfeit proceeds are voided. This risk is priced into the fee and bounded by diversification, but it is real and is addressed in detail in Section 7.

2. The Problem

Every Ark round creates a new covenant tree. The ASP funds the tree with BTC from its wallet. That BTC is frozen until the tree expires.

```
Round 1: ASP locks 10 BTC into Tree A     (expires in 7-30 days)
Round 2: ASP locks 10 BTC into Tree B     (expires in 7-30 days)
Round 3: ASP locks 10 BTC into Tree C     (expires in 7-30 days)
...
Trees overlap. All are locked simultaneously.
```

The lockup multiplier equals the expiry period divided by the capital cycling time. With more trees active at once, more BTC is frozen.

Table 1. Shipping defaults from source code (March 2026):

IMPLEMENTATION	VTXO EXPIRY	LOCKUP MULTIPLIER	SOURCE
Arkade (arkd)	7 days (`ARKD_VTXO_TREE_EXPIRY = 604672` sec)	~1.3× weekly throughput	`internal/config/config.go:241`
Bark (captaind)	30 days (`vtxo_lifetime = 4320` blocks)	~5.6× weekly throughput	`server/captaind.default.toml:14`

Both are operator-configurable. The choice is a tradeoff: shorter expiry is more capital-efficient but requires users to refresh more often. Longer expiry is better UX but locks more capital.

Table 2. Capital requirement by throughput:

ASP THROUGHPUT	CAPITAL NEEDED (7-DAY EXPIRY)	CAPITAL NEEDED (30-DAY EXPIRY)
100 BTC/week	~130 BTC	~560 BTC
500 BTC/week	~650 BTC	~2,800 BTC
1,000 BTC/week	~1,300 BTC	~5,600 BTC

At some throughput level — sooner for longer expiry, later for shorter — the ASP's own BTC is not enough.

3. The Cause

The capital gap is structural, not a bug. It comes from three properties of Ark's design:

1. **Trees overlap.** New trees are created every round while old trees haven't expired yet. The ASP must fund all active trees simultaneously.
2. **Expiry is the only recovery mechanism.** Under V1 (all shipping implementations), the ASP gets its BTC back only when the tree's absolute timelock expires. V2 revocation (proposed by Burak, not implemented, requires an unspecified ZK-based VPU) could enable early recovery. Until then, capital is locked for the full expiry period.
3. **Change outputs inflate the requirement.** When a user spends part of a VTXO, the remainder becomes a change output in the new tree — requiring additional capital from the ASP without generating additional fees.

Protocol-level improvements help but don't eliminate the gap. Shorter expiry reduces it. Delegation (Arkade) and hArk (Second) improve UX and throughput efficiency. But the multiplier is always greater than 1×. At meaningful throughput, external capital is needed.

4. The Solution

External LPs contribute BTC directly into Ark rounds. When a round's demand exceeds the ASP's available capital, the ASP solicits LP funding. The LP's UTXO enters the round's commitment transaction. The LP's repayment comes from forfeit proceeds on LP-funded VTXOs.

Figure 1. ArkFloat round flow:

```
LP                          ASP                         Ark Users
 |                           |                            |
 |                           |<-- round batch request ----|
 |                           |   (exceeds ASP capital)    |
 |                           |                            |
 |<-- liquidity request -----|                            |
 |   "Need X BTC, Y% fee"   |                           |
 |                           |                            |
 |--- UTXO for round ------->|                            |
 |                           |--- poolTx (includes LP) -->|
 |                           |   LP's UTXO in tree        |
 |                           |                            |
 |                      [users refresh VTXOs]             |
 |                           |                            |
 |<-- forfeit proceeds ------|                            |
 |   (paid to LP, not ASP)   |                            |
```

4.1 How It Works

1. ASP begins round construction. The batch exceeds available capital by X BTC.
2. ASP sends a `LiquidityRequest` to known LPs: amount needed, round details, proposed fee.
3. LP responds with a `LiquidityOffer`: UTXO to contribute, accepted rate, LP pubkey, forfeit recipient address.
4. ASP includes the LP's UTXO in the round's commitment transaction (`poolTx`).
5. Forfeit transactions for LP-funded VTXOs are constructed with the LP's address as recipient rather than the ASP's.
6. When users refresh into future rounds, their old VTXOs become claimable. The LP sweeps its share — receiving principal plus fee.

4.2 What the LP's Security Actually Is

The LP holds pre-signed Bitcoin transactions. These are valid on-chain regardless of ASP cooperation after the round is constructed. The LP can broadcast them at expiry without asking anyone's permission.

However, each forfeit transaction spends a specific VTXO output. That output only becomes a real UTXO when the user's leaf transaction is confirmed on-chain — either because the user exited unilaterally (broadcasting their path through the tree) or because the ASP sweeps at expiry. The LP's claim is therefore on forfeit proceeds from specific VTXOs in a specific round. It is not a general claim on the round's capital.

This distinction matters: the LP's recovery depends on how users in that round behave. Users who refresh into a new round make their old VTXOs available for the LP to claim via forfeit. Users who exit unilaterally

spend their VTXO through the exit path, which voids the forfeit. The LP gets nothing on VTXOs that exit.

- No external escrow
- No oracle
- No ASP cooperation required after round construction
- Recovery does depend on user behavior (refresh vs. unilateral exit)
- All terms in BTC, all timelocks in block heights

4.3 What the LP Needs to Understand

The LP's claim is proportional to the refresh rate in the funded round, not to the round's total capital. At a 5% unilateral exit rate on a 30 BTC LP position, the LP recovers ~28.5 BTC before fee. The fee is priced to cover expected exit rates.

The 2–5% figure is the base case. Exit rates are not random — they cluster around specific triggers. When an ASP misbehaves, raises fees sharply, or a competing ASP launches with better terms, exit rates can reach 30–60% within a single expiry period. The LP faces correlated tail risk: the same event that causes exits in one round typically affects all active rounds with that ASP simultaneously. Diversification across rounds with the same ASP does not reduce this tail risk.

The fee compensates for expected-case exits. The tail risk is managed by LP position sizing and multi-ASP diversification, not by the protocol.

4.4 Required Protocol Changes

Both Arkade and Bark would need the following changes. These are not minor additions — they touch the core round construction, forfeit construction, and verification paths in both implementations.

New message types

- `LiquidityRequest`: ASP to LP network. Contains round ID, amount needed, proposed fee, round expiry block height.
- `LiquidityOffer`: LP to ASP. Contains UTXO, accepted rate, LP pubkey, forfeit recipient address.

Modified round construction

- `poolTx` must accept external UTXOs from LPs alongside the ASP's own inputs. Currently both implementations fund the poolTx exclusively from the ASP wallet (Arkade: `builder.go:776–847`; Bark: `round/mod.rs:766–784`). Arkade has an existing boarding input path that may be repurposable; Bark does not.

- The tree's leaf structures must carry LP-funding metadata. Neither implementation currently supports per-leaf funding provenance (Arkade: flat leaf list at `builder.go:508`; Bark: `VtxoLeafSpec` at `tree/signed.rs:78–93`).

Modified forfeit construction

- Forfeit transactions must support per-VTXO recipient addresses.
- In Arkade: the low-level `BuildForfeitTx` primitive (`pkg/ark-lib/tree/forfeit_tx.go:9–22`) already accepts any script as a parameter. The calling layer (`VerifyForfeitTxs`, `getForfeitScript` at `builder.go:272–1186`) is hardcoded to the ASP's address and must be refactored to accept per-VTXO scripts.
- In Bark: the connector forfeit path (`lib/src/forfeit.rs:288–321`) hardcodes the server pubkey as output recipient and requires a function signature change. The hArk forfeit path (`lib/src/forfeit.rs:38–64`) uses a protocol-defined taproot encoding a musig sighash — changing the recipient changes the sighash, which requires users to co-sign a different transaction. The interaction between the hArk unlock preimage mechanism and a modified forfeit output is an open question (see Section 8).

Rounds without LP participation continue to work exactly as they do today. The changes are additive at the protocol level but require meaningful implementation work.

5. Impact

5.1 For ASPs

External capital removes the growth ceiling. An ASP with 300 BTC that would otherwise cap at 230 BTC/week (7-day expiry) or 55 BTC/week (30-day expiry) can serve whatever demand arrives, borrowing the difference from LPs. The borrowing cost (1–3% annualized on the LP-funded portion) is covered by transaction fees earned during the round.

The ASP doesn't need to choose between capital efficiency and user experience. It can run longer expiry (better UX, higher fees per refresh) and fund the larger capital requirement externally.

5.2 For LPs

A non-custodial BTC yield product. The LP's BTC goes into an Ark round, secured by pre-signed forfeit transactions. If the ASP vanishes, the forfeit transactions are still valid — the LP claims when VTXOs expire.

No platform custody. Counterparty risk is limited to actuarial exit risk.

The comparison is cold storage at 0%. At 1–3% annualized for a 7–30 day term with Bitcoin-script-enforced security, the product is competitive for any BTC holder who isn't actively deploying capital elsewhere — provided they understand that recovery is not guaranteed and scales with the refresh rate of the funded round.

5.3 For the Ark Ecosystem

The capital constraint is the binding limit on how many ASPs can operate and at what scale. Removing it means:

- Established ASPs can grow with demand instead of capping at their capital
- More competitive ASPs at scale means better fees for users and geographic diversity
- The "only Tether can afford to run an ASP at scale" scenario is avoided

ArkFloat solves the **throughput ceiling** problem: an ASP that already has users and fee revenue but cannot fund additional rounds. It does not solve the **bootstrapping problem**: getting a new ASP to critical mass in the first place. An ASP without throughput cannot generate the fee revenue to service LP borrowing costs — capital access without utilization is debt. The cold start problem is addressed in Section 7.4.

6. Viability

6.1 Implementation Comparison

Table 3. Current ASP implementation parameters (March 2026):

PARAMETER	ARKADE (ARKD)	BARK (CAPTAIND)	SOURCE
VTXO expiry default	604,672 sec (~7 days)	4,320 blocks (~30 days)	`config.go:241` / `captaind.default.toml:14`
Lockup multiplier	~1.3×	~5.6×	Derived
Round frequency	30s sessions, continuous	10s round interval	`config.go:228` / `captaind.default.toml:68`
Max VTXOs per round	128 participants	65,536 leaf VTXOs	config / `nb_round_nonces=8`, radix^8
Fee model	CEL expression programs (operator-set, zero default)	PPM tiers by VTXO age (0–0.8%)	`arkfee/README.md` / `captaind.default.toml:188`
Refresh fee near expiry	Operator-defined	~0% (0 ppm, 150 sat base)	`fees.rs`
Refresh fee on new VTXOs	Operator-defined	0.8% (8000 ppm)	`fees.rs`
Delegation	Yes (BIP322, shipping)	No	Arkade “Adios Expiry” feature
hArk (async rounds)	No	Yes (shipping)	bark v0.1.0-beta.6
V2 revocation	Not implemented	Not implemented	Blog post only (Burak, Medium)
Forfeit recipient parameterized	Low-level yes, calling layer no	No (hardcoded)	`forfeit_tx.go:9-22` / `forfeit.rs:314`

6.2 Fee Adequacy

The LP math clears at a blended user fee rate of ~0.35–0.40%. Whether ASPs can sustain that rate is a competitive question, not a technical one.

The relevant comparison is not Lightning routing fees. Lightning routing nodes earn fees on capital that both sides of the channel contribute — the routing node’s capital is matched by the peer’s. Ark ASPs front 100% of the locked capital unilaterally. That asymmetry justifies a structural premium.

More importantly, users paying Ark fees are buying a managed service: no node to run, no channel to open, no on-chain UTXOs required to receive payments. The implicit alternative for most Ark users is not “run a Lightning node at 0.01 sat/tx” — it is “don’t use Lightning at all.” The fee ceiling is what a user would pay to avoid self-custody infrastructure entirely, which is substantially higher than Lightning routing fees.

Bark’s fee schedule already supports 0.8% on VTXOs refreshed early (more than ~15 days before expiry). The blended rate is an operator choice. ASPs that compete on service quality — reliability, geographic

reach, compliance, customer support — rather than pure fee minimization can sustain rates that clear the LP spread.

At scale, on-chain economics improve further. OOR transactions between users on the same ASP have no on-chain footprint — one poolTx per round regardless of how many payments happen inside it. An ASP facilitating millions of daily transactions incurs the same on-chain construction cost as one facilitating ten. As transaction volume grows, the ASP's on-chain cost per transaction approaches zero, improving margin even at stable or declining fee rates. LP economics get progressively more viable as adoption grows. This benefit accrues disproportionately to high-volume ASPs, which is a further argument for LP capital concentration at established operators rather than new entrants.

6.3 When Is This Needed?

At 7-day expiry: ArkFloat becomes relevant when an ASP's throughput exceeds ~75% of its BTC holdings per week. An operator with 500 BTC can self-fund ~380 BTC/week. Beyond that, borrow or stop growing.

At 30-day expiry: ArkFloat becomes relevant much sooner. An operator with 500 BTC can self-fund only ~90 BTC/week.

The trigger is observable: ASPs report that capital constraints are limiting their throughput.

6.4 The Expiry Tradeoff

Operators choosing longer expiry (for better UX) need ArkFloat sooner but also earn more per unit of throughput — Bark's lockup-weighted fee tiers (up to 0.8%) compensate for the capital cost. The higher fee revenue makes borrowing costs more affordable.

Operators choosing shorter expiry (for capital efficiency) need ArkFloat later but earn less per transaction. They may never need it at moderate throughput.

Both strategies are viable. ArkFloat serves long-expiry ASPs first, then short-expiry ASPs as throughput scales.

7. Risks and Failure Modes

7.1 Unilateral Exit Risk (LP-specific)

This is the primary risk. If users in an LP-funded round exit unilaterally instead of refreshing, the LP's forfeit proceeds on those VTXOs are voided.

How unilateral exit works (verified in Bark source, `bark/src/exit/`): The exiting user broadcasts the transaction chain from the round TX root down to their own leaf. This confirms every intermediate node on their path on-chain. Other users in sibling subtrees are not affected — their branches remain off-chain and can still be swept by the ASP at expiry. The exiting user's VTXO is spent through the exit path, making the forfeit transaction for that VTXO unspendable.

Base case: Exits are expensive for users — multiple on-chain transactions with CSV delays at each tree level. Day-to-day exit rates of 1–3% are realistic.

Stress case: Exits cluster. The scenarios that drive exits above 5% are correlated with ASP distress:

SCENARIO	LIKELY EXIT RATE
Normal operation	1–3%
ASP raises fees significantly	10–20%
Competing ASP launches with better terms	20–40%
ASP misbehavior (attempted theft)	50–80%
ASP goes offline permanently	80–100%

When a stress event occurs, it typically affects all of the ASP's active rounds simultaneously. Diversifying across rounds with the same ASP does not protect against this tail risk — all positions are affected in the same event.

Fee pricing and the break-even problem: At a 5% expected exit rate, an LP lending 30 BTC needs ~1.5 BTC in fee just to break even on expected value. That is 5% of principal for a 14–30 day term. Annualized, this exceeds any realistic ASP fee revenue. The fee structure only works actuarially at scale — an LP running many rounds across multiple ASPs, where base-case exits average out and no single stress event dominates the portfolio.

> NOTE
> Single-round LP positions are speculative, not actuarial. The product requires LP scale and diversification to function as designed.

Mitigation options:

- LP diversifies across multiple ASPs (reduces correlation)
- LP caps position size per round (bounds single-event loss)
- ASP provides proof of reserves (UTXO ownership + tree expiry schedule) so LP can evaluate ASP health before lending
- Partial collateral posted by ASP to cover stress scenarios (a hybrid approach, outside current spec scope)

7.2 ASP Goes Offline After Round Construction

The LP's forfeit transactions are pre-signed and valid on-chain regardless of ASP operational status. The LP can broadcast them at expiry without the ASP. ASP cooperation is not required after the round is built.

7.3 On-Chain Fee Spikes

High fees affect Ark rounds (the ASP may reduce round frequency) but do not affect LP recovery. Forfeit transactions are pre-signed — the LP broadcasts them when fees are acceptable. At 500 sat/vB, a forfeit sweep costs a fraction of a percent of the LP's claim.

7.4 Cold Start

With 2 ASP implementations on mainnet and no established LP base, the market doesn't exist yet. The protocol changes need to be proposed, accepted by Ark Labs and/or Second, and implemented. This is a social and coordination challenge, not a technical one.

A subtler cold start problem affects small ASPs specifically. ArkFloat helps once an ASP is established — an ASP needs throughput to generate the fee revenue to afford LP borrowing costs, but throughput requires users, and users require an operational ASP that can fund rounds. The two conditions are circular. An ASP that borrows capital before reaching adequate utilization faces a debt service burden on top of an already-thin margin.

One mitigation available within the protocol's existing design: introductory LP relationships at below-market rates in exchange for priority deal flow as the ASP scales. An LP willing to accept 0.5–1% annual on an early-stage ASP secures preferential access to that ASP's deal flow at 2–3% once throughput is established. This is how Lightning Pool seeded its early market — initial channel leases were underpriced to build volume. The rate discovery mechanism (Open Question 4) should account for this tiering explicitly.

In practice, this mitigation may not function. Sophisticated LPs evaluating an early-stage ASP face a tail risk that dominates expected value: if the ASP fails before reaching throughput, the resulting mass unilateral exits void forfeit proceeds entirely. No introductory rate compensates for that outcome. Early LPs may

simply avoid small ASPs regardless of pricing, leaving the bootstrapping problem structurally unsolved by this protocol. An ASP-side partial collateral mechanism — the ASP posts collateral covering stress-scenario losses — is a more robust mitigation but is outside the current spec scope.

7.5 Over-Leverage

An ASP that borrows too much relative to its own capital and operational capacity could fail to maintain operations, triggering mass unilateral exits that harm LPs. Mitigation: LPs evaluate the ASP's balance sheet before lending. Proof of reserves (UTXO ownership attestation + tree expiry schedule) gives LPs visibility into the ASP's position.

8. Open Questions

1. **Bark hArk forfeit compatibility.** The hArk forfeit path in Bark uses a musig sighash computed over a protocol-defined output taproot. Changing the forfeit recipient changes the sighash, requiring users to co-sign a modified transaction. How does this interact with the hArk unlock preimage mechanism? Can an LP-addressed forfeit be constructed without breaking the hArk protocol guarantees?
2. **Arkade forfeit verification refactor scope.** `VerifyForfeitTxs` currently rebuilds every forfeit with the ASP's own script and rejects any that don't match. Supporting per-VTXO LP recipients requires per-VTXO script threading through round construction. What is the minimal change to the verification path?
3. **LP discovery.** How do ASPs find LPs? Initially bilateral (direct relationships). At scale, a bulletin board or batch auction modeled on Lightning Pool.
4. **Rate discovery.** What sets the lending rate? Initially bilateral negotiation. The natural band is above 0% (cold storage alternative) and below the ASP's fee revenue per unit of capital. In practice, 1–3% annualized is the likely range given Lightning liquidity market benchmarks (~2.6% on Amboss Magma, 1.2–1.7% on Voltage).
5. **Proof of reserves format.** What attestation of tree expiry schedules is sufficient for LPs to evaluate an ASP's capital position and leverage?
6. **Covenant interaction.** If Bitcoin adopts CTV or OP_CAT, richer VTXO scripts could encode LP claims directly as a spending condition — eliminating the need for modified forfeit construction entirely.
7. **Legal classification.** Is a BTC-for-BTC fixed-term loan structured as round participation a security, a swap, or something else?

9. References

ARK PROTOCOL

- Ark Protocol specification — VTXO mechanics, round structure, OOR payments
- Introduction to Ark — Shared UTXOs, forfeit mechanism, connectors
- VTXOs — Covenant trees, forfeit clause, expiry/refresh
- Connectors — Round atomicity, connector outputs
- OOR Payments — Arkoor mechanics, trust model
- Bitcoin Optech: Ark — Technical summary, newsletter coverage
- Ark Labs blog — Liquidity requirements, delegation ("Adios Expiry")
- Second blog — Liquidity research series (3 parts), yield survey, hArk
- Burak: Introducing Ark V2 — Revocation mechanism (not implemented)
- ben2077, issue #197 — Per-round external liquidity concept (this spec builds on it)

IMPLEMENTATIONS (SOURCE CODE)

- Arkade (arkd) (docs) — Go, 7-day default expiry, CEL fee programs, delegation
- Bark (captaind) (docs) — Rust, 30-day default expiry, PPM fee tiers, hArk async rounds

LIGHTNING LIQUIDITY MARKETS (PRECEDENT)

- Lightning Pool whitepaper — Batch auction architecture for channel leases
- Amboss Magma — P2P Lightning liquidity marketplace, ~2.6% APR median
- Voltage LINER data — 1.2–1.7% network yield benchmark

ADJACENT PROJECTS

- Lendasat — DLC VTXOs for user-facing lending on Ark
- Lygos Finance — Non-custodial DLC-based BTC lending

BITCOIN OPTECH COVERAGE

- V-PACK standard — Stateless VTXO verification (Mar 2026)
- hArk release — Second's async round implementation (Feb 2026)
- Arkade launches — Public beta (Nov 2025)
- OP_CHECKTEMPLATEVERIFY — Covenant proposal relevant to future LP claim scripting
- Covenants — Foundational dependency for richer VTXO scripts